# Loan Lifecycle Guardian

Project documentation: capabilities, architecture, rules, API, setup and developer reference

| | |
|---|---|
| Document date | 12 October 2026 |
| Version | 2.1 - checked against the current source files |
| Application | FastAPI + SQLAlchemy backend with a single-page HTML/JavaScript frontend |
| Status | Functional demo prototype. Not production banking software. |
| Data | All users, balances, loans, documents, scores and payments in this project are sample / simulated data. |
| Sources | Project README, Developer Guide and the application source code |

**Demo scope.** Credit scores, bank balances, decisions, document checks and payment processing are simulated. The application is meant for demonstrations, learning and prototyping with the included sample data.

## Contents


1. Overview and goals
2. Capabilities by role
3. Architecture and project structure
4. Domain model
5. Authentication and security model
6. User journeys
7. Business rules (EMI, eligibility, scoring, loan-fit)
8. Proactive Guardian alerts
9. AI assistant and prepayment simulation
10. Auto-pay engine
11. Document locker
12. API reference
13. Configuration
14. Setup, running and tests
15. Sample data and demo walkthrough
16. Extension points
17. Known limitations

# 1. Overview and goals

Loan Lifecycle Guardian simulates a modern digital lending workflow. Customers explore products, check eligibility, upload documents, apply for loans and manage repayments. Bank officers review applications, documents and service requests. Administrators manage users and the demo environment. An optional AI assistant gives role-aware guidance, and a proactive alert engine warns customers before an EMI problem becomes a missed payment.

The project demonstrates how a lending platform coordinates:

- customer onboarding and profile management
- product search, eligibility pre-checks and loan-fit ranking
- form validation and document verification flows
- officer approval, rejection and information requests
- repayment, auto-pay processing and failure diagnosis
- proactive, customer-specific servicing alerts
- notifications, service requests, role-based access control and auditing
- optional AI-written explanations, with all decisions made by deterministic rules

**Design principle.** Loan-fit ranking and eligibility decisions come from application rules. The optional language model only writes explanatory text. It does not approve loans, change rankings or move money.

# 2. Capabilities by role

## Customer

- Register (including phone, address, city and pincode), sign in, edit profile details and change password.
- Browse loan products, calculate an EMI, run eligibility pre-checks and view ranked loan-fit suggestions.
- Apply for a loan using the documents in the document locker.
- Upload PDF, PNG or JPG files. Checks cover content signature, size, duplicates and limited text-PDF verification; files that cannot be checked automatically are queued for officer review.
- View application status, loan schedule, payment history, accounts and notifications.
- Enable or disable auto-pay, pay an installment manually, download a CSV statement and request foreclosure.
- Receive proactive Guardian alerts for upcoming or overdue EMIs, auto-pay problems and applications that need more information. Quick actions are available from the Home page.
- Ask the Guardian assistant about loan status, auto-pay failures, next steps and prepayment what-if scenarios. Prepayment simulations are read-only.
- Submit customer support requests and password-reset requests.

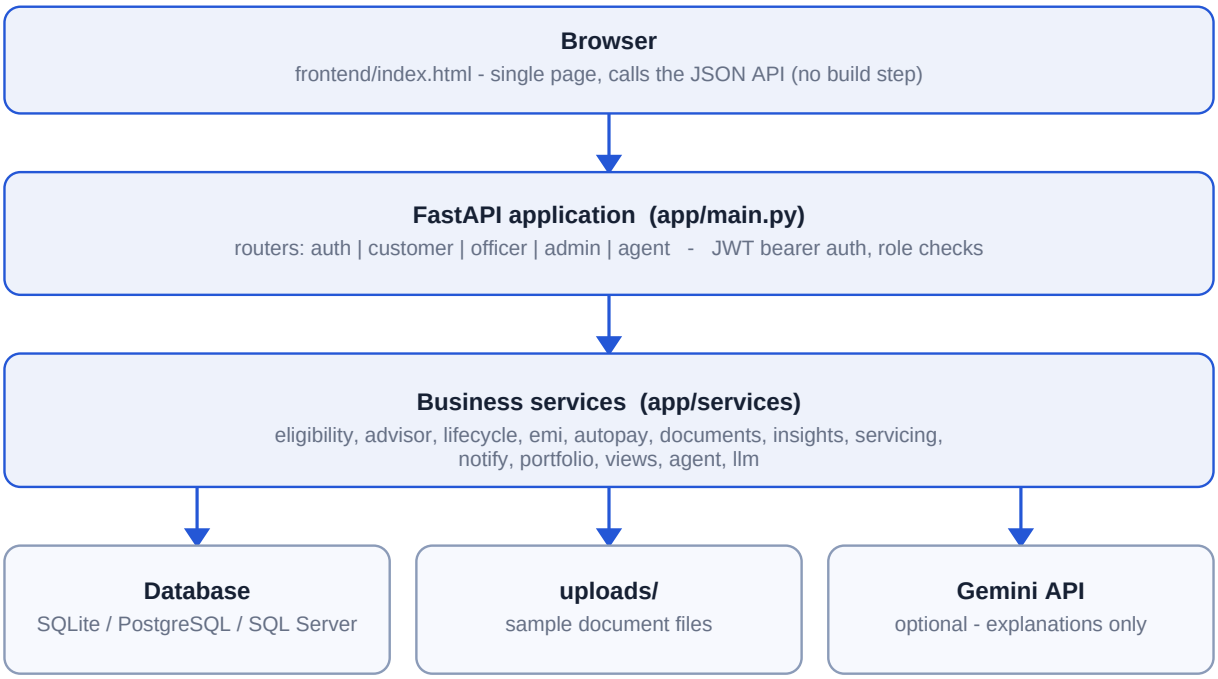
## Officer

- View and prioritise the application queue (ordered by the system recommendation).
- Inspect eligibility checks, the AI credit note and customer documents; request information; approve or reject.
- Verify or reject uploaded documents, update KYC, send payment reminders and handle service requests.
- View the portfolio dashboard and collection summaries.

## Administrator

- Everything an officer can do.
- Manage staff and users, enable or disable accounts and initiate customer password resets.
- Review password-reset requests, freeze or unfreeze demo accounts, run the auto-pay engine and inspect audit events.

## 3. Architecture and project structure




| Area | Responsibility |
|---|---|
| app/main.py | Application initialisation, startup table creation, optional demo seeding, static frontend, health endpoint, optional auto-pay scheduler. |
| app/config.py | Environment-variable configuration (also reads .env). |
| app/database.py | SQLAlchemy engine, session factory and request-scoped sessions. |
| app/models.py, app/schemas.py | Database entities; request and response validation models. |
| app/security.py | Password hashing, JWT creation and validation, role checks. |
| app/seed.py | Loan products and the sample users, loans, applications and documents. |
| app/routers/ | API routes grouped by actor: auth, customer, officer, admin, agent. |
| app/services/ | Business logic: eligibility, advisor, lifecycle, EMI, documents, auto-pay, insights, servicing, notifications, portfolio, views, agent and LLM wrapper. |
| frontend/index.html | Browser UI and API client; no separate frontend build is needed. |
| db/ | migration_add_service_requests.sql: one SQL Server migration for the service-request table. It is not a complete migration system. |
| docs/ | DEVELOPER_GUIDE.md. |
| tests/ | conftest.py, test_core.py, test_features.py (endpoint and business-logic tests). |

### Repository layout

```
loanlifecycle-guardian/
  app/
    __init__.py  config.py  database.py  main.py  models.py  schemas.py  security.py  seed.py
    routers/   admin.py  agent.py  auth.py  customer.py  officer.py  __init__.py
    services/  advisor.py  agent.py  autopay.py  documents.py  eligibility.py  emi.py  errors.py
               insights.py  lifecycle.py  llm.py  notify.py  portfolio.py  servicing.py  views.py
  db/        migration_add_service_requests.sql
  docs/      DEVELOPER_GUIDE.md
  frontend/  index.html
  tests/     __init__.py  conftest.py  test_core.py  test_features.py
  .env.example  README.md  requirements.txt  requirements-extra.txt  requirements-dev.txt  run.py
```

## 4. Domain model

| Entity | Purpose and key fields |
|---|---|
| User | username (unique), full name, email, password hash, role (CUSTOMER, OFFICER, ADMIN), active flag. |
| CustomerProfile | Date of birth, PAN, monthly income, EMIs to other lenders, simulated credit score, employment type, KYC flag, phone, address, city, pincode, employer. |
| BankAccount | Simulated account number, balance, net-banking flag, frozen flag, daily debit limit. |
| LoanProduct | Amount and tenure limits, base rate, minimum income and score, maximum FOIR and LTV, maximum age at maturity, required documents. |
| LoanApplication | A proposed loan: amount, tenure, computed rate, EMI and FOIR, system score and recommendation, check results, AI note, officer decision. Open statuses: SUBMITTED, NEED_INFO. |
| Loan | An issued loan: principal, rate, tenure, EMI, outstanding, status (ACTIVE, CLOSED), auto-pay flag, mandate status and limit. |
| Installment | One scheduled EMI: due date, EMI / principal / interest parts, status (PENDING, PAID, FAILED, OVERDUE, CLOSED_EARLY), attempts, last failure code. |
| Payment | Payment events and failure codes: DISBURSAL, AUTOPAY, MANUAL, FORECLOSURE. |
| Document | Locker entry: type, stored name, hash, status (UPLOADED, VERIFIED, REJECTED), source (UPLOAD or SAMPLE), automated check results, reviewer note. |
| Notification | Inbox messages and alerts. |
| ServiceRequest | Customer support requests handled by officers. |
| PasswordResetRequest | Customer request reviewed by an administrator. |
| AuditLog | Actor, action, entity and detail for key actions and state changes. |

## 5. Authentication and security model

### JWT flow

- The app issues a JWT on login or registration. It carries the user id, the role and an expiry.
- A bearer token is required for protected endpoints. get_current_user() validates it and loads the user; require_roles(...) restricts a route to specific roles.
- Every request re-checks that the user exists and is active, so disabling an account takes effect immediately.

### Passwords

Passwords are never stored in plain text. The code uses PBKDF2-HMAC-SHA256 with 200,000 iterations, a random per-user salt and a constant-time comparison. Login attempts are throttled: five failures for the same (username, IP) lock further attempts for five minutes.

### Authorisation rules

- Customers can view and manage only their own profile, loans, documents, applications and requests. Requests for another customer's record return 404.
- Officers act on application queues, document reviews and support requests.
- Administrators additionally manage users, accounts, password resets, auto-pay runs and audit access.
- Customer routes accept only CUSTOMER tokens, officer routes accept OFFICER and ADMIN, admin routes accept ADMIN. A request without a token receives 401; the wrong role receives 403.

### Other safeguards

- Pydantic validation on every request (username, PAN, phone, pincode patterns; password strength; age 18-80).
- PAN values are masked in responses.
- Key actions are written to the audit log (logins, decisions, document views and reviews, password events, account freezes, engine runs). Temporary passwords are never written to the log.
- Uploaded files are stored outside the static folder and served with X-Content-Type-Options: nosniff.

## 6. User journeys

### A. Registration and login

A user posts to /api/auth/register. The app creates a User with role CUSTOMER, a CustomerProfile and a default BankAccount, and returns a JWT. The registration form also collects phone (10 digits), address, city and pincode (6 digits); they are optional on the API and are stored on the profile. /api/auth/login validates credentials and returns a token.

### B. Product browsing and eligibility

Customers list active products (/api/products), calculate EMIs (/api/emi/calc), run a pre-check (/api/me/applications/precheck) and open the loan-fit ranking (/api/me/loan-fit) and the proactive alerts (/api/me/insights).

### C. Application creation

The request is validated by Pydantic, the eligibility engine runs, and the application is stored as SUBMITTED with its system score and recommendation. Documents are read from the customer's locker, and officers are notified. A customer cannot hold two open applications for the same product.

### D. Document handling

Uploads are checked for content signature, size and duplicates, and text PDFs are compared with the profile. Reviewer states are UPLOADED (pending review), VERIFIED and REJECTED. Anything the automatic checks cannot read goes to officer review.

### E. Officer review and decision

The officer opens the queue, reviews checks and documents, and approves, rejects or requests more information. The decision re-validates against the latest profile, documents and loans and locks the application so two officers cannot decide it at once. Approval is refused while required documents are still awaiting officer verification (the override flag does not bypass this), and approving a REJECT or NEED_DOCS case needs an override flag plus a justification note.

### F. Servicing and repayment

After approval the system disburses the loan, creates the installment schedule and, if enabled, collects EMIs through auto-pay. Customers can pay manually, toggle auto-pay, prepay or foreclose. Failed or overdue payments are tracked and notifications are raised.

### G. Agent assistance

/api/agent/chat connects a signed-in user to the assistant. It explains loan status, alerts, auto-pay issues and prepayment scenarios within the demo rules. State-changing requests are staged and require a separate confirmation message.

# 7. Business rules

## 7.1 EMI

```
EMI = P * r * (1+r)^n / ((1+r)^n - 1)      r = annual_rate / 1200,  n = months
```

A zero rate uses P / n. The amortisation schedule rounds every row to two decimals and lets the last instalment absorb the rounding residue, so the principal parts sum exactly to the loan amount.

## 7.2 Loan products (sample configuration)

| Product | Amount (Rs) | Tenure (mo) | Base rate | Min income | Min score | Max FOIR | Max LTV |
|---|---|---|---|---|---|---|---|
| HOME | 5 L - 2 Cr | 60-360 | 8.5% | 30,000 | 700 | 50% | 80% |
| PERSONAL | 50 K - 25 L | 12-60 | 10.5% | 20,000 | 700 | 50% | - |
| AUTO | 1 L - 30 L | 12-84 | 9.0% | 25,000 | 680 | 50% | 85% |
| EDUCATION | 1 L - 50 L | 12-180 | 9.5% | 15,000 | 650 | 60% | - |
| GOLD | 20 K - 20 L | 3-36 | 9.0% | 10,000 | 600 | 70% | 75% |

## 7.3 Interest rate by credit score

| Credit score | Rate applied |
|---|---|
| 800 and above | base - 0.50 points |
| 750 to 799 | base rate |
| 700 to 749 | base + 0.75 points |
| below 700 | base + 1.50 points |

## 7.4 Eligibility checks

Hard checks: amount range, tenure range, age (minimum 21; age at maturity within the product limit), minimum income, credit score, repayment capacity (FOIR) and, for secured products, a declared asset value and loan-to-value. Document-class checks: KYC verified and required documents received. Soft warnings: score within 30 points of the cut-off, FOIR within 5 points of the limit, and documents still awaiting officer verification.

**FOIR** = (EMIs to other lenders + EMIs of the customer's active loans at this bank + new EMI) / monthly income. The bank's own loans are read from the database, so they cannot be under-declared.

## 7.5 Scoring and recommendation

```
score = 100 - 30*(hard failures) - 10*(document / KYC failures) - 8*(soft warnings)     (minimum 0)
```

| Condition | System recommendation |
|---|---|
| Any hard failure | REJECT |
| Document or KYC gap (no hard failure) | NEED_DOCS |
| Soft warning, or score below 80 | REVIEW |
| Otherwise | APPROVE |

## 7.6 Loan-fit advisor

For each product the advisor computes the maximum principal that fits inside the FOIR headroom, lists blockers (age, score, income, headroom below the product minimum) and, when none apply, an ease score:

```
ease = 40 + min(25, margin_over_cutoff / 4) + min(25, free_capacity_ratio * 60)
       - 8*soft_warnings - 10 (KYC pending) + 10 (all documents present) - 10 (secured product)
likelihood: HIGH >= 75, MEDIUM >= 55, otherwise LOW
```

Credit bands: 800+ Excellent, 750-799 Very good, 700-749 Good, 650-699 Fair, below 650 Needs work. The result is an estimate, not an approval. The language model may narrate it but cannot change it.

# 8. Proactive Guardian alerts

The insights engine (app/services/insights.py) is what makes the assistant proactive rather than purely reactive. GET /api/me/insights returns customer-specific alerts, sorted from most to least urgent. The Home page renders them as cards with action buttons, and the assistant can read the same data through its get_my_insights tool.

| Situation (per active loan) | Severity | Alert and actions |
|---|---|---|
| EMI failed, overdue or past its due date | high | "EMI #n overdue" with the amount, due date and the diagnosed reason. Actions: Pay now; Add funds (demo) when the reason is insufficient balance. |
| Due within 7 days, auto-pay OFF | medium | Warns the EMI is close. Actions: Enable auto-pay, Pay now. |
| Due within 7 days, auto-pay ON but the debit would fail | high | "Auto-debit on [date] will FAIL" with the reason and first fix step. Actions depend on the reason: Add funds (demo), Enable net banking (demo), Reset mandate limit. |
| Due within 7 days, auto-pay ON and healthy | info | Confirms the debit is set up and which account is used. |
| Due later, auto-pay OFF | low | Suggests turning auto-pay on. Action: Enable auto-pay. |
| Application needs more information | high | Shows the officer's note. Action: Open applications. |
| Nothing to report | info | "All good - no action needed right now." |

The Home page offers payment, auto-pay and account actions that reuse the normal customer endpoints. Actions marked demo (adding funds, enabling net banking) work only while DEMO_MODE=1, and every action needs the customer to click it. Alerts are computed on request from live loan, installment and account state, so they always reflect the current situation.

# 9. AI assistant and prepayment simulation

## 9.1 Tools and roles

- With a Gemini key, the assistant uses function calling. Tools are closures bound to the signed-in user, so the model can only reach that user's data.
- Customer tools (12 in total) cover loan products, EMI calculation, the customer's loans, auto-pay diagnosis, eligibility, best-loan ranking, applications, the proactive alerts (**get_my_insights**) and the read-only **simulate_prepayment**, plus three staged actions: apply, enable auto-pay, disable auto-pay.
- Officers and administrators get read-only tools for the queue and application checks; they only suggest, and the decision stays in the officer console.
- Text returned by tools (for example a loan purpose) is declared to the model as data, not instructions.

## 9.2 Confirmation gate for actions

apply_for_loan, enable_autopay and disable_autopay do not execute when the model calls them. They only stage the action and return a summary. The action runs only when the customer's own next message is an explicit yes (a no cancels it). A staged action expires after ten minutes, and a yes with nothing pending is treated as an ordinary message. The model cannot confirm on the customer's behalf.

### 9.3 Read-only prepayment simulation

The simulate_prepayment tool answers questions such as "what happens if I pay Rs 50,000 extra?". It is customer-scoped (a loan that is not the customer's returns "Loan not found") and non-mutating: it never changes the loan or moves funds. It reuses the servicing logic:

| Input | Result |
|---|---|
| Amount below the outstanding balance | PART_PREPAYMENT: the new outstanding balance, plus two options compared side by side. **Reduce EMI** keeps the remaining months and lowers the EMI; **Reduce tenure** keeps the EMI and shortens the term. Each shows the resulting EMI, months and interest saved. |
| Amount at or above the outstanding balance | FORECLOSURE: the payable amount and the interest saved. |
| Loan not active, or amount not positive | A clear error message. |

### 9.4 Offline fallback

Without a model or key, a rule-based fallback answers common questions: best loan, auto-pay failure, EMI and due dates, application status, and prepayment what-if prompts. The fallback recognises prepayment questions before generic EMI advice, so a user asking "what if I pay extra" gets a simulation instead of a generic EMI answer. The application is fully usable offline.

## 10. Auto-pay engine

The engine runs when an administrator presses Run auto-pay, or on a timer when AUTOPAY_SCHEDULER_SECONDS is greater than zero. It collects due installments oldest-first, one loan at a time, and takes row locks where the database supports them so concurrent runs cannot debit the same installment twice. If auto-pay is off the customer receives a reminder. After MAX_AUTOPAY_ATTEMPTS failures an installment is skipped unless the administrator ticks "ignore attempt limit".

When a debit would fail, these conditions are checked **in this order** and the first one that applies is reported:

| # | Code | Meaning and customer guidance |
|---|---|---|
| 1 | NO_ACCOUNT | No linked account; link a savings account. |
| 2 | ACCOUNT_FROZEN | Account is frozen; visit the branch. |
| 3 | MANDATE_NOT_ACTIVE | Auto-pay is off or the mandate is inactive; enable and authorise. |
| 4 | NETBANKING_DISABLED | Net banking is switched off; a product-specific menu path is shown. |
| 5 | MANDATE_LIMIT_EXCEEDED | Mandate limit is lower than the EMI; raise it. |
| 6 | DAILY_LIMIT_EXCEEDED | EMI is above the account's daily debit limit. |
| 7 | INSUFFICIENT_BALANCE | Balance is below the EMI; the exact shortfall is shown. |

Every explanation includes a product-specific net-banking path and a product-specific note. Successful debits update the balance, installment, outstanding principal and payment ledger, and close the loan after the last installment.

## 11. Document locker

- **Accepted files:** PDF, PNG, JPG up to MAX_UPLOAD_MB (default 5). The type is detected from the file signature, so a renamed file is rejected.
- **Duplicates:** the same file for the same document type is refused; the same file under another type is flagged.
- **Automatic verification:** text-based PDFs are parsed. The name must match the profile, an ID proof must contain the registered PAN and an income proof must be within 15% of the declared income. If all pass the document is VERIFIED.

- **Manual review:** images, scans and unreadable PDFs become UPLOADED (pending). Officers verify or reject; a reason is mandatory for rejection. Every officer view and review is audited.
- **Storage:** UPLOAD_DIR/<user_id>/<generated name> with a path-traversal guard.
- **Isolation:** a customer requesting another customer's document receives 404.
- **Sample documents:** with DEMO_MODE=1 the Documents page offers a Use sample button that generates a clearly labelled sample PDF from the customer's profile. The seeded demo customers also receive sample documents.
- **Document types:** identity proof, address proof, income proof, property papers, vehicle quotation, admission letter and gold valuation certificate. Each product lists the types it requires.

## 12. API reference

All routes live under /api. Send `Authorization: Bearer <token>` on protected routes. The complete generated OpenAPI schema is at /docs while the app is running.

### 12.1 Public and shared

| Method | Path | Purpose |
|---|---|---|
| POST | /api/auth/register | Create a customer, profile (with optional phone, address, city, pincode) and bank account. |
| POST | /api/auth/login | Returns a JWT and the user. |
| GET | /api/auth/me | Current user. |
| POST | /api/auth/password-reset-requests | Customer password-reset request; identical reply whether or not the user exists. |
| GET | /api/products | Active loan products. |
| POST | /api/emi/calc | EMI, total interest, total payable. |
| POST | /api/agent/chat | Role-aware assistant. |
| GET | /api/health | Service status and whether Gemini is configured. |

### 12.2 Customer (role CUSTOMER)

| Method | Path | Purpose |
|---|---|---|
| GET | /api/me/insights | Proactive Guardian alerts with actions. |
| GET / POST | /api/me/accounts[/demo] | Accounts; demo sandbox switches (DEMO_MODE). |
| GET | /api/me/loans | Loans with next due, progress and overdue count. |
| GET | /api/me/loans/{id}/schedule | Installment schedule. |
| GET | /api/me/loans/{id}/autopay-status | Diagnosis of the next debit (explain=true adds AI text). |
| POST | /api/me/loans/{id}/autopay | Enable or disable auto-pay. |
| POST | /api/me/loans/{id}/pay | Pay the next EMI manually. |
| POST | /api/me/loans/{id}/foreclose | Pay the full outstanding and close. |
| GET | /api/me/loans/{id}/payments | Payment ledger. |
| GET | /api/me/loans/{id}/statement.csv | CSV statement. |
| POST | /api/me/applications/precheck | Run eligibility without submitting. |
| POST / GET | /api/me/applications | Submit / list applications. |
| POST | /api/me/applications/{id}/resubmit | Re-queue after NEED_INFO. |
| GET / PUT | /api/me/profile | View (PAN masked) / update profile. |

| Method | Path | Purpose |
|---|---|---|
| POST | /api/me/password | Change password. |
| GET | /api/me/loan-fit | Ranked products (explain=true adds narrative). |
| GET / POST | /api/me/documents | List the locker / upload a file. |
| POST | /api/me/documents/sample | Generate a sample document (DEMO_MODE). |
| GET / DELETE | /api/me/documents/{id}[/file] | Download or delete an own document. |
| GET / POST | /api/me/notifications[/read] | List / mark all read. |
| GET / POST | /api/me/requests | Support requests (a linked loan id must be positive and owned by the customer). |

### 12.3 Officer (roles OFFICER, ADMIN)

| Method | Path | Purpose |
|---|---|---|
| GET | /api/officer/applications | Queue ordered APPROVE, REVIEW, NEED_DOCS, REJECT. |
| GET | /api/officer/applications/{id} | Detail with checks, documents and AI note. |
| POST | /api/officer/applications/{id}/decision | approve / reject / need_info. |
| POST | /api/officer/applications/{id}/summary/refresh | Regenerate the AI note. |
| POST | /api/officer/applications/{id}/draft-message | AI draft of a customer message. |
| POST | /api/officer/customers/{id}/kyc | Mark KYC verified. |
| GET | /api/officer/customers/{id} | Customer overview. |
| GET | /api/officer/documents/{id}/file | View a document (audited). |
| POST | /api/officer/documents/{id}/review | Verify or reject a document. |
| GET | /api/officer/dashboard | Portfolio KPIs. |
| GET | /api/officer/collections | Due and overdue installments. |
| POST | /api/officer/loans/{id}/remind | Send a payment reminder. |
| GET | /api/officer/requests | Open service requests. |
| POST | /api/officer/requests/{id}/respond | Respond, resolve or reject. |

### 12.4 Administrator (role ADMIN)

| Method | Path | Purpose |
|---|---|---|
| GET / POST | /api/admin/users | List users / create staff. |
| POST | /api/admin/users/{id}/toggle | Enable or disable a user. |
| POST | /api/admin/users/{id}/reset-password | Reset a customer password; returns a temporary one. |
| GET | /api/admin/password-reset-requests | List reset requests. |
| POST | /api/admin/password-reset-requests/{id}/review | Approve or reject a request. |
| GET | /api/admin/accounts | Customer accounts. |
| POST | /api/admin/accounts/{id}/freeze | Freeze or unfreeze an account. |
| POST | /api/admin/run-autopay | Run the engine (as_of, ignore_attempt_limit). |
| GET | /api/admin/audit | Latest 100 audit events. |

# 13. Configuration

All settings are read from process environment variables and, when present, a local .env file. Copy .env.example to .env and edit it.

| Variable | Default | Description |
|---|---|---|
| DATABASE_URL | sqlite:///./loan_guardian.db | SQLAlchemy database URL. SQLite is the local default. |
| JWT_SECRET | development secret in code | Signing secret for session tokens. Set a unique, long random value outside a local demo. With DEMO_MODE=0 startup refuses the built-in development secret, the .env.example placeholder and values shorter than 32 characters. |
| JWT_EXPIRE_MIN | 120 | JWT lifetime in minutes. |
| DEMO_MODE | 1 | Enables demo behaviour and sample-document tooling. |
| SEED_DEMO | 1 | Seeds sample records when the database has no users. Set 0 to disable. |
| GEMINI_API_KEY / GOOGLE_API_KEY | unset | Optional Gemini credential; either variable works. |
| GEMINI_MODEL | gemini-3.8-flash | Gemini model name (see app/config.py). |
| AUTOPAY_SCHEDULER_SECONDS | 0 | 0 disables the background scheduler; a positive value is the interval in seconds. |
| MAX_AUTOPAY_ATTEMPTS | 5 | Maximum simulated auto-debit attempts per installment. |
| UPLOAD_DIR | ./uploads | Local storage path for documents. |
| MAX_UPLOAD_MB | 5 | Maximum accepted document size. |

Generate a secret with:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Set variables for the current PowerShell session, for example:

```
$env:JWT_SECRET = "use-a-unique-long-random-value-here"
$env:DEMO_MODE = "0"
$env:SEED_DEMO = "0"
```

### Database URL examples

```
# SQLite (default)
DATABASE_URL=sqlite:///./loan_guardian.db

# PostgreSQL (install psycopg2-binary)
DATABASE_URL=postgresql+psycopg2://user:password@host/database?sslmode=require

# SQL Server (install pyodbc and a Microsoft ODBC driver); the value is one line in .env
DATABASE_URL=mssql+pyodbc://@localhost\SQLEXPRESS/LoanGuardian
    ?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes&TrustServerCertificate=yes
```

# 14. Setup, running and tests

### Requirements

Python 3.10 or later. SQLite ships with Python and is the default database.

| Package | Purpose |
|---|---|
| fastapi | HTTP API and request handling |

| Package | Purpose |
|---|---|
| uvicorn | ASGI web server |
| pydantic | Request and data validation |
| SQLAlchemy | ORM and database connections |
| PyJWT | JWT authentication tokens |
| python-dotenv | Loads the local .env file |
| python-multipart | Multipart document uploads |

| Optional package (requirements-extra.txt) | Install when |
|---|---|
| google-genai | Gemini explanations and the function-calling assistant are wanted. Without it or a key the app uses rule-based responses. |
| pyodbc | SQL Server is selected (also needs a Microsoft ODBC Driver). |
| psycopg2-binary | PostgreSQL is selected. |

requirements-dev.txt includes the core requirements plus httpx (FastAPI's test client) and pytest. The files declare minimum versions rather than a locked environment; add a lock file if byte-for-byte reproducibility is required.

### Run locally on Windows

```
py -3.10 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements-dev.txt
copy .env.example .env        # optional; defaults use SQLite and need no Gemini key
python run.py
```

Open http://127.0.0.1:8000 for the app, /docs for the API reference and /api/health for a status check. On first startup SQLAlchemy creates missing tables, and with demo seeding enabled it creates the sample products, users, applications, loans and documents when the user table is empty. create_all does not alter existing tables, so back up and migrate a database before changing model columns. For a disposable local demo you can delete the local database file and restart to reseed from scratch.

### Tests

```
python -m pytest -q
```

Tests use a separate SQLite database (test_guardian.db) and upload directory (test_uploads/). They cover EMI calculations, role authorisation, validation, loan decisions, auto-pay outcomes, document checks and review, customer isolation, loan-fit ranking, service requests, confirmation-gated agent actions and read-only, customer-scoped prepayment simulations. Approval is rejected while required documents await officer verification, and linked service-request loan ids must be positive and owned by the customer.

## 15. Sample data and demo walkthrough

When SEED_DEMO=1 the app creates sample accounts for demonstrations. All values are fictional.

| Role | Username | Password | Situation illustrated |
|---|---|---|---|
| Administrator | admin | Admin@123 | Runs the auto-pay engine, manages users |
| Officer | officer1 | Officer@123 | Reviews the application queue |
| Customer | rahul | Cust@1234 | Auto-pay fails: insufficient balance |
| Customer | priya | Cust@1234 | Auto-pay fails: net banking disabled |
| Customer | suresh | Cust@1234 | Auto-pay succeeds |

| Role | Username | Password | Situation illustrated |
|---|---|---|---|
| Customer | anita | Cust@1234 | Open application; system recommends APPROVE |
| Customer | vikram | Cust@1234 | Open application; system recommends REJECT |

The sample loans have their first EMI due on the day of seeding, so running the auto-pay engine immediately shows each outcome.

### Suggested walkthrough

| Step | What to show |
|---|---|
| 1 | Sign in as admin, open Admin and press Run auto-pay. The results list shows PAID for suresh and the failure codes for rahul and priya. |
| 2 | Sign in as rahul. The Home page shows a Guardian alert for the failed EMI. Use Add funds (demo), then Pay now. |
| 3 | Open the Guardian assistant and ask "why did my auto-pay fail?" and "what if I pay Rs 50,000 extra?" to see a read-only prepayment comparison. |
| 4 | Ask the assistant to turn auto-pay off. It stages the action and asks for a yes; reply yes to confirm. |
| 5 | Sign in as priya to see the net-banking diagnosis and the product-specific steps. |
| 6 | Sign in as officer1. Open the queue (anita is APPROVE, vikram is REJECT), review the AI credit note and checks, verify a document and decide. |

## 16. Extension points

- Add loan products through the LoanProduct seed data.
- Expand notifications in services/notify.py.
- Add document-check rules in services/documents.py.
- Add queue management and reporting in the officer logic.
- Add new insight rules in services/insights.py; the Home page renders any alert the endpoint returns.
- Add assistant tools in services/agent.py (keep state-changing tools behind the confirmation gate).
- Swap the demo LLM for a more controlled enterprise assistant through services/llm.py.

## 17. Known limitations

- Credit score calculation is a simulation; income and external obligations are self-reported.
- Document verification is intentionally limited: it checks file signatures and simple uncompressed text PDFs, not OCR, document authenticity or official KYC records. Other files need manual review.
- Loan balances and payment execution are simulated. No real bank, payment network, bureau or mandate provider is connected.
- Monetary values use floating-point fields; use decimal / numeric types for financial-grade calculations.
- The auto-pay scheduler and pending assistant confirmations are in memory and designed for a single process.
- Manual payment and auto-pay use row locks where the database supports them; SQLite does not provide equivalent row-level locking, so the prototype is not designed for concurrent or multi-worker use.
- Foreclosure is based on outstanding principal; accrued interest within the current month is not modelled in the live routes.
- Login throttling is in memory and should be replaced with shared persistent rate limiting for multi-worker deployment.
- The unauthenticated password-reset request endpoint is de-duplicated while a request is pending but has no IP or account rate limit; add shared throttling before public deployment.

- Password changes and resets update the stored password, but issued JWTs stay valid until they expire (120 minutes by default) unless the account is deactivated. Token revocation or session versioning would be needed beyond a demo.
- Sample documents and the fixed demo credentials are not production records or secure defaults.
- There is no general migration framework: schema changes on an existing database need manual SQL or Alembic.
- Production use would additionally require secret management, persistent storage, KYC / OCR integration, observability, deployment hardening and independent security and financial review.